

Analisis del código P1

- Analisis del código C

▼ `server.c`

▼ CÓDIGO

```
#include <stdio.h>
#include <stdlib.h> // Para exit, atoi y otras funciones estándar
#include <string.h> // Para bzero y otras funciones de cadenas
#include <unistd.h> // Para read, write y otras funciones de E/S
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

void error(char *msg)
{
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[])
{
    int sockfd, newsockfd, portno;
    socklen_t clilen;
    char buffer[256];
    struct sockaddr_in serv_addr, cli_addr;
    int n;

    // ===== Configuración y Conexión ===== //
```

```

/* 1. Validar argumentos (programa + puerto)
*/
if (argc < 2) {
    fprintf(stderr, "ERROR, no port provided\n");
    exit(1);
}

/* 2. Obtener el puerto de los argumentos */
portno = atoi(argv[1]);

/* 3. Creación del socket */
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)
    error("ERROR opening socket");

/* 4. Reutilización del puerto (Opcional) */
int optval = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

/* 5. Configurar la dirección del servidor (serv_addr) */
bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = INADDR_ANY;
serv_addr.sin_port = htons(portno);

/* 6. Vincula el socket a la dirección del servidor (serv_addr) */
if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
    error("ERROR on binding");

```

```

    /* 7. Habilita el Socket para escuchar conexio
nes (hasta 5 en la cola de espera) */
    listen(sockfd, 5);

    /* 8. Bloquearse esperando un cliente */
    clilen = sizeof(cli_addr);
    newsockfd = accept(sockfd, (struct sockaddr *)
&cli_addr, &clilen);
    if (newsockfd < 0)
        error("ERROR on accept");

    // ===== Comunicación =====
    //

    /* 9. Leer los datos del cliente */
    bzero(buffer, 256);
    n = read(newsockfd, buffer, 255);
    if (n < 0)
        error("ERROR reading from socket");
    printf("Here is the message: %s\n", buffer);

    /* 10. Envía una respuesta al cliente */
    n = write(newsockfd, "I got your message", 18);
    if (n < 0)
        error("ERROR writing to socket");

    /* 11. Cierra los sockets */
    close(newsockfd);
    close(sockfd);

    return 0;

```

```
}
```

- El **flujo del código** es el siguiente...

Configuración y Conexión...

▼ 1. Validar argumentos (programa + puerto)

```
if (argc < 2) {  
    fprintf(stderr, "ERROR, no port provided  
\\n");  
    exit(1);  
}
```

- `argc` y `argv` :
 - `argc` es la cantidad de argumentos de línea de comandos, incluyendo el nombre del programa.
 - `argv[0]` es el nombre del programa (`./server`).
 - `argv[1]` es el puerto en el que el servidor va a escuchar.
 - A diferencia del cliente, el servidor **no recibe hostname**, porque no se conecta a nadie: solo espera conexiones entrantes en su propio puerto.
- **Validación:**
 - si no se pasa al menos 1 argumento además del nombre del programa (es decir, si `argc < 2`), se muestra `"ERROR, no port provided"` por `stderr` y se sale con `exit(1)`.
 - El umbral es `2` (y no `3` como en el cliente) porque el servidor solo necesita el puerto, no el host.

▼ 2. Obtener el puerto de los argumentos

```
portno = atoi(argv[1]);
```

- `portno = atoi(argv[1])` :

- convierte la cadena del puerto (`argv[1]`) a entero y lo guarda en `portno` .
- Ese valor se usará más adelante en `htons(portno)` para completar `serv_addr.sin_port` .

▼ 3. Creación del socket

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)
    error("ERROR opening socket");
```

- **Función:** `int socket(int domain, int type, int protocol);`
- **Qué hace:** crea un endpoint de comunicación, pero **todavía no está conectado a ningún servidor**. Solo reserva los recursos locales.
- **Parámetros:**
 - `domain` : `AF_INET` indica IPv4.
 - `type` : `SOCK_STREAM` indica TCP (flujo de bytes, orientado a conexión).
 - `protocol` : `0` para que el sistema elija el protocolo adecuado (TCP en este caso).
- **Valor de retorno:** un descriptor de archivo (`sockfd`). Si falla, devuelve `1` y se llama a `error()` .

▼ 4. Reutilización del puerto

```
int optval = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval,
sizeof(optval));
```

- `setsockopt()` (opcional pero útil):
 - `setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval))`
 - Permite reutilizar el puerto local aunque queden conexiones en estado `TIME_WAIT` .

- Evita el error "Address already in use" al reiniciar el servidor.

▼ 5. Configurar la dirección del servidor (serv_addr)

```
bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = INADDR_ANY;
serv_addr.sin_port = htons(portno);
```

- `bzero()` : limpia la estructura `serv_addr` para que no queden valores basura.
- `serv_addr.sin_family = AF_INET` : indica que usamos IPv4.
- `serv_addr.sin_addr.s_addr = INADDR_ANY` : el servidor escuchará en **todas las interfaces de red** de la máquina.
- `serv_addr.sin_port = htons(portno)` : convierte el número de puerto al orden de bytes de red (big-endian).

▼ 6. Vincula el socket a la dirección del servidor (serv_addr)

```
if (bind(sockfd, (struct sockaddr *) &serv_addr,
sizeof(serv_addr)) < 0)
    error("ERROR on binding");
```

- `bind()` :
 - **Prototipo:** `int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
 - **Qué hace:**
 - asocia el socket `sockfd` con la dirección IP y el puerto especificados.
 - A partir de aquí, el socket "existe" en ese puerto.
 - Si otro proceso ya usa ese puerto, `bind()` falla.
 - **Parámetros:**
 - `sockfd` : descriptor del socket creado en el paso 1.

- `addr`: puntero a la estructura con la dirección y puerto. Se castea a `struct sockaddr *` porque `bind()` es genérica.
- `addrlen`: tamaño de la estructura (`sizeof(serv_addr)`).
- **Valor de retorno:** `0` si tuvo éxito, `1` si falla.

▼ 7. Poner el Socket a escuchar conexiones

```
listen(sockfd, 5);
```

- **Función:** `int listen(int sockfd, int backlog);`
- **Qué hace:**
 - convierte el socket en **pasivo**, es decir, queda a la espera de conexiones entrantes.
 - Sin `listen()`, el socket no acepta conexiones TCP aunque se haya hecho `bind()`.
- **Parámetros:**
 - `sockfd`: descriptor del socket ya asociado a una dirección con `bind()`.
 - `backlog`: cantidad máxima de conexiones pendientes que el sistema operativo puede encolar mientras el servidor no llama a `accept()`. En este ejemplo es `5`.
- **Valor de retorno:** `0` si tuvo éxito, `1` si falla.

▼ 8. Bloquearse a la espera del cliente.

```
clilen = sizeof(cli_addr);
newsockfd = accept(sockfd, (struct sockaddr *) &
cli_addr, &clilen);
if (newsockfd < 0)
    error("ERROR on accept");
```

- **Función:** `int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);`

- **Qué hace:** `accept()` es **bloqueante**. El proceso servidor se queda dormido hasta que un cliente intenta conectarse. Cuando eso ocurre, extrae una conexión de la cola de `listen()` y devuelve un nuevo socket conectado a ese cliente. El socket original (`sockfd`) sigue existiendo y sigue escuchando para futuras conexiones, pero en este ejemplo el servidor solo atiende una conexión y luego termina.
- **Parámetros:**
 - `sockfd` : descriptor del socket de escucha (el que se puso en `listen()`).
 - `addr` : puntero a una estructura donde `accept()` guardará la dirección del cliente que se conectó. Puede ser `NULL` si no nos interesa.
 - `addrlen` : puntero a un `socklen_t` . Antes de llamar debe contener el tamaño de la estructura `addr` ; después de la llamada, el sistema lo actualiza con el tamaño real de la dirección del cliente. Por eso se usa `&clilen` .
- **Valor de retorno:** un **nuevo descriptor de socket** (`newsockfd`) para comunicarse con ese cliente. Si falla, devuelve `-1` .
- **Importante:** `newsockfd` es el que se usa para `read()` y `write()` , no `sockfd` .

Comunicación...

▼ 9. Recibir datos del cliente

```
bzero(buffer, 256);
n = read(newsockfd, buffer, 255);
if (n < 0)
    error("ERROR reading from socket");
printf("Here is the message: %s\n", buffer);
```

- **Función:** `ssize_t read(int fd, void *buf, size_t count);`
- **Qué hace:** lee datos que el cliente envió a través de la conexión TCP. Es una llamada **bloqueante**: si no hay datos disponibles, el proceso se queda esperando.

- **Parámetros:**

- `fd`: descriptor del socket conectado (`newsockfd`).
- `buf`: buffer donde se guardarán los datos leídos (`buffer`).
- `count`: cantidad máxima de bytes a leer. Aquí `255` para dejar lugar al terminador nulo `\0` si se imprime como cadena.

- **Valor de retorno:** cantidad de bytes efectivamente leídos. Puede ser menor a `count`. Devuelve `0` si el cliente cerró la conexión, o `1` si hay error.

- **En el código:** se limpia el buffer con `bzero` y luego se lee. El mensaje se muestra por pantalla con `printf`.

▼ 10. Envía una respuesta al cliente

```
n = write(newsockfd, "I got your message", 18);
if (n < 0)
    error("ERROR writing to socket");
```

- **Función:** `ssize_t write(int fd, const void *buf, size_t count);`

- **Parámetros:**

- `fd`: descriptor del socket conectado (`newsockfd`).
- `buf`: datos a enviar. En este caso, la cadena literal `"I got your message"`.
- `count`: cantidad de bytes a escribir. Se usa `18` porque es la longitud exacta de la cadena sin el `\0`.

- **Valor de retorno:** cantidad de bytes efectivamente escritos. Puede ser menor a `count`. Devuelve `1` si hay error.

- **Qué hace:** envía datos al cliente a través de la conexión TCP. Es bloqueante si el buffer de envío del socket está lleno.

- **En el código:** el servidor responde siempre con el mismo mensaje fijo, sin importar lo que haya recibido.

▼ 11. Cierra la conexión.

```
close(newsockfd);  
close(sockfd);
```

- **Función:** `int close(int fd);`
- **Qué hace:** cierra el descriptor y libera los recursos asociados. En TCP, al cerrar un socket conectado se envía un `FIN` al otro extremo para iniciar la terminación de la conexión.
- **Parámetro:** `fd`: descriptor de archivo o socket a cerrar.
- **Valor de retorno:** `0` si tuvo éxito, `1` si falla.
- **En el código:**
 - `close(newsockfd)`: cierra la conexión con el cliente.
 - `close(sockfd)`: cierra el socket de escucha, liberando el puerto.
- **Importante:** el orden no es crítico, pero cerrar ambos es necesario para liberar todos los recursos. Si no se cierra, el descriptor queda ocupado hasta que el proceso termine.

▼ `client.c`

▼ CÓDIGO

```
#include <stdio.h>  
#include <sys/types.h>  
#include <sys/socket.h>  
#include <netinet/in.h>  
#include <netdb.h>  
#include <string.h> // Para strlen y otras funciones de cadenas  
#include <unistd.h> // Para read, write y otras funciones de E/S  
#include <stdlib.h> // Para exit y otras funciones estándar
```

```

void error(char *msg)
{
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[])
{
    int sockfd, portno, n;
    struct sockaddr_in serv_addr;
    struct hostent *server;
    char buffer[256];

    // ===== Configuración y Co
nexión ===== //

    /* 1. Validar argumentos (programa + host + pue
rto) */
    if (argc < 3) {
        fprintf(stderr, "usage %s hostname port\n", a
rgv[0]);
        exit(1);
    }

    /* 2. Obtener el puerto */
    portno = atoi(argv[2]);

    /* 3. Crear Socket */
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0)
        error("ERROR opening socket");

    /* 4. Resolver el nombre del host */

```

```

server = gethostbyname(argv[1]);
if (server == NULL) {
    fprintf(stderr, "ERROR, no such host\n");
    exit(0);
}

/* 5. Configurar la dirección del servidor (serv_addr) */
bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
bcopy((char *)server->h_addr,
      (char *)&serv_addr.sin_addr.s_addr,
      server->h_length);
serv_addr.sin_port = htons(portno);

/* 6. Conectarse al servidor */
if (connect(sockfd, (struct sockaddr *)&serv_addr,
            sizeof(serv_addr)) < 0)
    error("ERROR connecting");

/* 7. Leer mensaje por teclado */
printf("Please enter the message: ");
bzero(buffer, 256);
fgets(buffer, 255, stdin);

// ===== Comunicación =====
//

/* 8. Envía mensaje al servidor */
n = write(sockfd, buffer, strlen(buffer));
if (n < 0)
    error("ERROR writing to socket");

```

```

/* 9. Espera recibir una respuesta */
bzero(buffer, 256);
n = read(sockfd, buffer, 255);
if (n < 0)
    error("ERROR reading from socket");
printf("%s\n", buffer);

/* 10. Muestra la respuesta y Cierra la conexión */
close(sockfd);

return 0;
}

```

- El **flujo del código** es el siguiente...

Configuración y Conexión

▼ 1. Validar argumentos

```

if (argc < 3) {
    fprintf(stderr, "usage %s hostname port\n",
    argv[0]);
    exit(1);
}
portno = atoi(argv[2]);

```

- **argc** y **argv** :
 - **argc** es la cantidad de argumentos de línea de comandos.
 - **argv[0]** es el nombre del programa,
 - **argv[1]** el hostname del servidor y

- `argv[2]` el puerto.

- **Validación:**

- si no se pasan al menos 3 argumentos (programa + host + puerto), se muestra un mensaje de uso y se sale con `exit(1)`.

▼ 2. Obtener el puerto

- `atoi(argv[2])`:
 - convierte la cadena del puerto a entero.
 - Se guarda en `portno`.
- **Importante:** no se valida que el puerto sea positivo o que el host exista; eso se maneja más adelante.

▼ 3. Crear socket con `socket()`

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);  
if (sockfd < 0)  
    error("ERROR opening socket");
```

- **Función:** `int socket(int domain, int type, int protocol);`
- **Qué hace:** crea un endpoint de comunicación, pero **todavía no está conectado a ningún servidor**. Solo reserva los recursos locales.
- **Parámetros:**
 - `domain`: `AF_INET` indica IPv4.
 - `type`: `SOCK_STREAM` indica TCP (flujo de bytes, orientado a conexión).
 - `protocol`: `0` para que el sistema elija el protocolo adecuado (TCP en este caso).
- **Valor de retorno:** un descriptor de archivo (`sockfd`). Si falla, devuelve `1` y se llama a `error()`.

▼ 4. Resolver el nombre del host con `gethostbyname()`

```
server = gethostbyname(argv[1]);
if (server == NULL) {
    fprintf(stderr, "ERROR, no such host\n");
    exit(0);
}
```

- **Función:** `struct hostent *gethostbyname(const char *name);`
- **Parámetro:** `name` es el nombre del host (por ejemplo, `"localhost"` o `"192.168.1.10"`). En el código es `argv[1]`.
- **Valor de retorno:** puntero a una estructura `struct hostent` con información del host (dirección IP, alias, tipo de dirección, etc.). Si no se puede resolver, devuelve `NULL`.
- **Qué hace:** traduce un nombre de host a una dirección IP. En este ejemplo se usa la versión obsoleta `gethostbyname()`, pero es la que aparece en el tutorial clásico.
- **Nota:** si el host no existe, el programa termina con `exit(0)`. Debería ser `exit(1)`, pero el código provisto lo deja así.

▼ 5. Configurar la dirección del servidor (`serv_addr`)

```
// 4a. limpia la estructura del socket 'serv_addr'
bzero((char *) &serv_addr, sizeof(serv_addr));

// 4b. indica que la dirección es IPv4
serv_addr.sin_family = AF_INET;

// 4c. Copia la dirección IP y el puerto del servidor a la estructura del socket 'serv_addr'
bcopy((char *)server->h_addr,
      (char *)&serv_addr.sin_addr.s_addr,
      server->h_length);
serv_addr.sin_port = htons(portno);
```

- `bzero()` : limpia la estructura `serv_addr` para evitar valores basura. Equivale a `memset(..., 0, ...)`.
- `serv_addr.sin_family = AF_INET` : indica que la dirección es IPv4.
- `bcopy()` : copia la dirección IP resuelta por `gethostbyname()` desde `server->h_addr` hacia `serv_addr.sin_addr.s_addr`.
 - `server->h_addr` es un puntero a la primera dirección IP de la lista.
 - `server->h_length` es la longitud de esa dirección (normalmente 4 bytes para IPv4).
 - `bcopy` es similar a `memcpy`, pero con orden de argumentos invertido (origen, destino, tamaño). Está obsoleta, pero se usa en el ejemplo.
- `serv_addr.sin_port = htons(portno)` : convierte el puerto al orden de bytes de red (big-endian). `htons` significa "host to network short".
- **Resultado:** `serv_addr` queda lista con la IP y el puerto del servidor.

▼ 6. Conectarse al servidor con `connect()`

```
if (connect(sockfd, (struct sockaddr *)&serv_addr,
    sizeof(serv_addr)) < 0)
    error("ERROR connecting");
```

- **Función:** `int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
- **Qué hace:** inicia la conexión TCP con el servidor. Si el servidor no está escuchando (`listen()` no ejecutado), `connect()` falla. Si el servidor está escuchando pero no llamó a `accept()`, la conexión puede completarse igual porque el sistema operativo la encola en el backlog. Esto es clave para el experimento 2.b.
- **Parámetros:**
 - `sockfd` : descriptor del socket creado en el paso 2.

- `addr`: puntero a la estructura con la dirección del servidor. Se castea a `struct sockaddr *` porque `connect()` es genérica.
- `addrlen`: tamaño de la estructura (`sizeof(serv_addr)`).
- **Valor de retorno:** `0` si la conexión se establece, `1` si falla.
- **Bloqueante:** `connect()` puede bloquearse hasta que se establece la conexión o se produce un error.

▼ 7. Leer mensaje por teclado

```
printf("Please enter the message: ");
bzero(buffer, 256);
fgets(buffer, 255, stdin);
```

- `printf()`: muestra el prompt por pantalla.
- `bzero(buffer, 256)`: limpia el buffer para asegurarse de que no queden datos viejos.
- `fgets(buffer, 255, stdin)`:
 - **Función:** `char *fgets(char *s, int size, FILE *stream);`
 - **Qué hace:** lee una línea de texto. Incluye el `\n` si entra en el buffer. Termina la cadena con `\0`.
 - **Parámetros:**
 - `s`: buffer donde se guarda la cadena.
 - `size`: cantidad máxima de caracteres a leer (incluyendo el `\0`). Aquí 255.
 - `stream`: `stdin` para leer desde teclado.

Comunicación...

▼ 8. Envía el mensaje al servidor

```
n = write(sockfd, buffer, strlen(buffer));
if (n < 0)
    error("ERROR writing to socket");
```

- `write(sockfd, buffer, strlen(buffer))`:
 - **Función:** `ssize_t write(int fd, const void *buf, size_t count)`;
 - **Parámetros:**
 - `fd`: socket conectado (`sockfd`).
 - `buf`: datos a enviar (`buffer`).
 - `count`: cantidad de bytes a enviar. Se usa `strlen(buffer)` para no enviar el `\0` final.
 - **Valor de retorno:** cantidad de bytes escritos. Puede ser menor a `count`. Si es `1`, hay error.
 - **Qué hace:** envía el mensaje al servidor a través de la conexión TCP.

▼ 9. Recibir la respuesta del servidor

```
bzero(buffer, 256);

n = read(sockfd, buffer, 255);
if (n < 0)
    error("ERROR reading from socket");
```

- `bzero(buffer, 256)`: limpia el buffer antes de leer la respuesta.
- `read(sockfd, buffer, 255)`:
 - **Función:** `ssize_t read(int fd, void *buf, size_t count)`;
 - **Qué hace:** espera y lee la respuesta del servidor. Es bloqueante.
 - **Parámetros:**
 - `fd`: socket conectado (`sockfd`).
 - `buf`: buffer donde se guardará la respuesta (`buffer`).
 - `count`: máximo de bytes a leer (255).
 - **Valor de retorno:** cantidad de bytes leídos. Puede ser menor a `count`. Devuelve `0` si el servidor cerró la

conexión, `1` si hay error.

▼ 10. Muestra la respuesta y cierra la conexión

```
printf("%s\n", buffer);  
close(sockfd);
```

- `printf("%s\n", buffer)` : muestra la respuesta por pantalla.
Como el servidor envía `"I got your message"` sin `\0`, se asume que el buffer ya tenía ceros o que la cadena termina correctamente. En la práctica, podría imprimir basura si no se limpia bien, pero el ejemplo funciona.
- `close(sockfd)` :
 - **Qué hace:** cierra la conexión TCP, libera el descriptor y envía un `FIN` al servidor.
 - **Función:** `int close(int fd);`
 - **Parámetro:** descriptor del socket a cerrar.

• **Análisis del código Java**

▼ `Server.java`

▼ **CÓDIGO**

- El **flujo del código** es el siguiente...

Configuración y Conexión

▼ 1. Validar argumentos (programa + puerto)

```
if ((args.length != 1) || (Integer.valueOf(args  
[0]) <= 0) )  
{  
    System.out.println("1 arguments needed: por
```

```
t");
    System.exit(1);
}
```

- `args` es el arreglo de argumentos de línea de comandos en Java. Equivale a `argv` en C.
 - `args.length` es la cantidad de argumentos (sin contar el nombre del programa, a diferencia de C).
 - `args[0]` es el puerto.
- **Validación doble:**
 - `args.length != 1` : debe haber **exactamente** 1 argumento (el puerto). Si hay 0 o más de 1, error.
 - `Integer.valueOf(args[0]) <= 0` : el puerto debe ser un número positivo. Si no lo es, error.
- **Salida:** si falla la validación, se imprime `"1 arguments needed: port"` y se sale con `System.exit(1)`.
- **Diferencia con C:** en C se usa `argc < 2` (al menos 1 argumento además del nombre del programa). En Java, `args.length` ya no cuenta el nombre del programa, así que se compara contra `1` directamente.

▼ 2. Crear el `ServerSocket` (socket + bind + listen)

```
ServerSocket serverSocket = null;
try
{
    serverSocket = new ServerSocket(Integer.valueOf(args[0]));
}
catch (Exception e)
{
    System.out.println("Error on server socket");
    System.exit(1);
}
```

- **Clase `ServerSocket`** : representa el socket de escucha del servidor. Equivale a las llamadas `socket()` + `bind()` + `listen()` de C.
- **Constructor `new ServerSocket(int port)`** :
 - Crea el socket.
 - Hace `bind()` al puerto indicado en todas las interfaces locales (`0.0.0.0`).
 - Hace `listen()` con un backlog por defecto (normalmente 50).
- **Diferencia clave con C:** en C estas tres operaciones son llamadas separadas y podés hacer una sin la otra. En Java, el constructor las hace **todas juntas**. No hay forma directa de tener un `ServerSocket` con `bind()` pero **sin** `listen()`. Por eso el experimento 2.a no se puede reproducir igual en Java.
- **Manejo de errores:** si el puerto está ocupado o hay otro problema, el constructor lanza una excepción (`IOException` o `BindException`), que se captura y se reporta con `"Error on server socket"`.
- `serverSocket` se usará luego solo para `accept()` y para cerrarse al final. No se usa para leer ni escribir datos.

▼ 3. Bloquearse a la espera del cliente con `accept()`

```
Socket connected_socket = null;

try
{
    connected_socket = serverSocket.accept();
}
catch (IOException e)
{
    System.err.println("Error on Accept");
    System.exit(1);
}
```

- **Método `serverSocket.accept()`** :

- Es **bloqueante**: el hilo se queda esperando hasta que llegue una conexión.
- Devuelve un **nuevo objeto** `Socket` (`connected_socket`) que representa la conexión con ese cliente en particular.
- Equivale a `accept()` en C, que devuelve un nuevo file descriptor.
- `connected_socket` : es el socket que se usará para leer y escribir datos. El `serverSocket` original sigue existiendo y sigue escuchando, pero en este ejemplo no se vuelve a usar (el servidor atiende una sola conexión y termina).
- **Manejo de errores**: si `accept()` falla, se lanza `IOException` , se imprime `"Error on Accept"` y se sale.
- **Importante**: en Java, el `ServerSocket` ya hizo `listen()` en su constructor, así que las conexiones pueden encolarse antes de que se llame a `accept()` . Esto permite reproducir el experimento 2.b igual que en C.

▼ 4. Obtener los streams de entrada/salida

```
DataInputStream fromclient;
DataOutputStream toclient;

fromclient = new DataInputStream(connected_socket.getInputStream());
toclient    = new DataOutputStream(connected_socket.getOutputStream());
```

- **Diferencia clave con C**: en C se lee y escribe **directamente** sobre el file descriptor del socket con `read()` y `write()` . En Java, **no se puede**. Siempre hay que pasar por **streams** asociados al socket.
- `connected_socket.getInputStream()` : devuelve un `InputStream` para leer datos que llegan del cliente.
- `connected_socket.getOutputStream()` : devuelve un `OutputStream` para enviar datos al cliente.

- `DataInputStream` / `DataOutputStream` : son envoltorios que agregan métodos cómodos para leer/escribir tipos primitivos y arreglos de bytes.
- `fromclient` : stream de entrada, usado luego con `read(buffer)` .
- `toclient` : stream de salida, usado luego con `write(buffer, 0, length)` .

▼ 5. Recibir datos del cliente con `read()`

```
byte[] buffer;
buffer = new byte[256];

fromclient.read(buffer);

String str = new String(buffer);

System.out.println("Here is the message: " + str);
```

- `buffer = new byte[256]` : se reserva un arreglo de 256 bytes. Equivale al `char buffer[256]` de C.
- `fromclient.read(buffer)` :
 - **Método:** `int read(byte[] b)` de `DataInputStream` (heredado de `InputStream`).
 - **Parámetro:** `b` es el arreglo donde se guardan los bytes leídos.
 - **Valor de retorno:** cantidad de bytes leídos, o `1` si el stream llegó al final. En este ejemplo **no se captura el valor de retorno**, lo cual es un descuido: podría haber leído menos de 256 bytes y el resto quedaría con ceros.
 - **Qué hace:** lee datos del cliente. Es bloqueante si no hay datos disponibles.
- `new String(buffer)` : convierte el arreglo de bytes a una cadena usando la codificación por defecto de la plataforma. Equivale a tratar el buffer como string en C.

- `System.out.println(...)` : imprime el mensaje recibido por pantalla, igual que el `printf` del servidor C.

▼ 6. Enviar una respuesta al cliente con `write()`

```
String strresp = "I got your message";
System.out.println("strrsp " + strresp);

buffer = strresp.getBytes();

toclient.write(buffer, 0, buffer.length);
```

- `String strresp = "I got your message"` : se define la respuesta fija, igual que en C.
- `System.out.println("strrsp " + strresp)` : impresión de depuración (no esencial, pero aparece en el código).
- `buffer = strresp.getBytes()` : convierte la cadena a un arreglo de bytes. Equivale a preparar el buffer que se enviará con `write()` en C.
- `toclient.write(buffer, 0, buffer.length)` :
 - **Método:** `void write(byte[] b, int off, int len)` de `DataOutputStream` (heredado de `OutputStream`).
 - **Qué hace:** envía los bytes al cliente a través de la conexión TCP.
 - **Parámetros:**
 - `b` : arreglo de bytes a enviar.
 - `off` : offset desde dónde empezar a enviar (`0`).
 - `len` : cantidad de bytes a enviar (`buffer.length`, que es 18).
 - **Diferencia con C:** en C se pasa el puntero al buffer y la cantidad de bytes. En Java se pasa el arreglo, un offset y la longitud.

▼ 7. Cerrar streams, socket conectado y `ServerSocket`

```
fromclient.close();
toclient.close();
connected_socket.close();
serverSocket.close();
```

- `fromclient.close()` y `toclient.close()` : cierran los streams de entrada y salida. En Java es importante cerrarlos para liberar recursos y hacer flush de los buffers.
- `connected_socket.close()` : cierra la conexión con el cliente. Envía un `FIN` al otro extremo.
- `serverSocket.close()` : cierra el socket de escucha y libera el puerto.
- **Orden:** primero los streams, luego el socket conectado, luego el `ServerSocket` . Es el orden recomendado porque los streams dependen del socket.
- **Diferencia con C:** en C solo se cierra el file descriptor del socket. En Java hay que cerrar también los streams porque son objetos independientes que envuelven al socket.

▼ `Client.java` y `Client2.java`

▼ CÓDIGO

```
/*
 * Client.java
 * Just sends stdin read data to and receives back
 some data from the server
 *
 * usage:
 * java Client serverhostname port
 */

import java.io.*;
```

```

import java.net.*;

public class Client
{
    public static void main(String[] args) throws IOException
    {

        /* 1. Validar argumentos (host + puerto) */
        if ((args.length != 2) || (Integer.valueOf(args
[1]) <= 0) )
        {
            System.out.println("2 arguments needed: server
hostname port");
            System.exit(1);
        }

        /* 2. Crear el Socket y conectarse al servidor
        */
        Socket socketwithserver = null;
        try
        {
            socketwithserver = new Socket(args[0], Integer
r.valueOf(args[1]));
        }
        catch (Exception e)
        {
            System.out.println("ERROR connecting");
            System.exit(1);
        }

        /* 3. Obtener los streams de entrada/salida */
        DataInputStream fromserver;
        DataOutputStream toserver;
        fromserver = new DataInputStream(socketwithserv

```

```

er.getInputStream());
    toserver = new DataOutputStream(socketwithserver.getOutputStream());

    /* 4. Leer mensaje del usuario por consola */
    Console console = System.console();
    String inputline = console.readLine("Please enter the message: ");

    /* 5. Enviar el mensaje al servidor */
    byte[] buffer;
    buffer = inputline.getBytes();
    toserver.write(buffer, 0, buffer.length);

    /* 6. Enviar el mensaje al servidor */
    buffer = new byte[256];
    fromserver.read(buffer);
    String resp = new String(buffer);
    System.out.println(resp);

    /* 7. Cerrar streams y socket */
    fromserver.close();
    toserver.close();
    socketwithserver.close();

    }
}

```

```

/*
 * Client2.java
 * Just sends stdin read data to and receives back
 some data from the server
 */

```

```

* usage:
* java Client serverhostname port
*/

import java.io.*;
import java.net.*;

public class Client2
{
    public static void main(String[] args) throws IOException
    {

        /* 1. Validar argumentos (host + puerto) */
        if ((args.length != 2) || (Integer.valueOf(args
[1]) <= 0) )
        {
            System.out.println("2 arguments needed: server
hostname port");
            System.exit(1);
        }

        /* 2. Crear el Socket y conectarse al servidor
        */
        Socket socketwithserver = null;
        try
        {
            socketwithserver = new Socket(args[0], Integer
r.valueOf(args[1]));
        }
        catch (Exception e)
        {
            System.out.println("ERROR connecting");
            System.exit(1);
        }
    }
}

```

```

        /* 3. Obtener los streams de entrada/salida */
        DataInputStream fromserver;
        DataOutputStream toserver;
        fromserver = new DataInputStream(socketwithserv
er.getInputStream());
        toserver = new DataOutputStream(socketwithser
ver.getOutputStream());

        /* 4. Leer mensaje del usuario */
        BufferedReader console = new BufferedReader(ne
w InputStreamReader(System.in));
        System.out.print("Please enter the message: ");
        String inputline = console.readLine();

        /* 5. Enviar el mensaje al servidor */
        byte[] buffer;
        buffer = inputline.getBytes();
        toserver.write(buffer, 0, buffer.length);

        /* 6. Enviar el mensaje al servidor */
        buffer = new byte[256];
        fromserver.read(buffer);
        String resp = new String(buffer);
        System.out.println(resp);

        /* 7. Cerrar streams y socket */
        fromserver.close();
        toserver.close();
        socketwithserver.close();

    }
}

```

- El `Client.java` y el `Client2.java` son **casi idénticos**. La única diferencia está en **cómo leen el mensaje del usuario**:

- `Client.java` usa `System.console()`.
- `Client2.java` usa `BufferedReader` sobre `System.in`.

- El **flujo del código** es el siguiente...

Configuración y Conexión...

▼ 1. Validar argumentos (host + puerto)

```
if ((args.length != 2) || (Integer.valueOf(args
[1]) <= 0) )
{
    System.out.println("2 arguments needed: serv
erhostname port");
    System.exit(1);
}
```

- `args` es el arreglo de argumentos de línea de comandos. Equivale a `argv` en C, pero **sin contar el nombre del programa**.
 - `args[0]` es el hostname del servidor.
 - `args[1]` es el puerto.
- **Validación doble:**
 - `args.length != 2`: debe haber **exactamente** 2 argumentos (host + puerto). Si hay menos o más, error.
 - `Integer.valueOf(args[1]) <= 0`: el puerto debe ser un número positivo.
- **Salida:** si falla la validación, se imprime `"2 arguments needed: serverhostname port"` y se sale con `System.exit(1)`.
- **Diferencia con el servidor Java:** el servidor solo necesita 1 argumento (el puerto), el cliente necesita 2 (host + puerto).
- **Diferencia con C:** en C se valida `argc < 3` porque `argc` cuenta el nombre del programa. En Java, `args.length` no lo cuenta, así

que se compara contra `2`.

▼ 2. Crear el Socket y conectarse al servidor

```
Socket socketwithserver = null;
try
{
    socketwithserver = new Socket(args[0], Integer.valueOf(args[1]));
}
catch (Exception e)
{
    System.out.println("ERROR connecting");
    System.exit(1);
}
```

- **Clase `Socket`**: representa el socket del cliente, ya conectado al servidor. Equivale a las llamadas `socket()` + `connect()` de C, pero **en un solo constructor**.
- **Constructor `new Socket(String host, int port)`**:
 - **host**: `args[0]`, el nombre o IP del servidor.
 - **port**: `args[1]` convertido a `int` con `Integer.valueOf()`.
 - **Qué hace**: resuelve el nombre del host a IP (equivalente a `gethostbyname()` en C), crea el socket (equivalente a `socket()`), y se conecta al servidor (equivalente a `connect()`). Todo junto.
- **Diferencia clave con C**: en C tenés que hacer `socket()`, `gethostbyname()`, configurar `serv_addr`, y recién después `connect()`. En Java, el constructor `new Socket(host, port)` hace todo eso automáticamente.
- **Manejo de errores**: si la conexión falla (servidor no escucha, host inexistente, etc.), se lanza una excepción que se captura y se reporta con `"ERROR connecting"`.
- **socketwithserver**: es el socket conectado. Se usará para obtener los streams de entrada/salida.

▼ 3. Obtener los streams de entrada/salida

```
DataInputStream fromserver;  
DataOutputStream toserver;  
fromserver = new DataInputStream(socketwithserver.  
getInputStream());  
toserver = new DataOutputStream(socketwithserv  
er.getOutputStream());
```

- **Diferencia clave con C:** en C se lee y escribe **directamente** sobre el file descriptor del socket con `read()` y `write()`. En Java, **siempre** hay que pasar por streams asociados al socket.
- `socketwithserver.getInputStream()` : devuelve un `InputStream` para leer datos que llegan del servidor.
- `socketwithserver.getOutputStream()` : devuelve un `OutputStream` para enviar datos al servidor.
- `DataInputStream` / `DataOutputStream` : envoltorios que agregan métodos cómodos para leer/escribir bytes y tipos primitivos.
- `fromserver` : stream de entrada, usado luego con `read(buffer)`.
- `toserver` : stream de salida, usado luego con `write(buffer, 0, length)`.

▼ 4. Leer mensaje del usuario

```
Console console = System.console();  
String inputline = console.readLine("Please ente  
r the message: ");
```

- `System.console()` : devuelve un objeto `Console` que representa la consola del sistema. Permite leer y escribir de forma interactiva.
- `console.readLine("Please enter the message: ")` :
 - **Método:** `String readLine(String fmt, Object... args)`.

- **Parámetro:** `fmt` es el prompt que se muestra antes de leer (en este caso, `"Please enter the message: "`).
- **Valor de retorno:** la línea leída como `String`, sin el `\n` final.
- **Qué hace:** muestra el prompt y espera a que el usuario escriba una línea y presione Enter.

```
BufferedReader console = new BufferedReader(new
InputStreamReader(System.in));
System.out.print("Please enter the message: ");
String inputline = console.readLine();
```

- **Diferencia importante con `Client2.java`:** `Client2` usa `BufferedReader` sobre `System.in`, que es más portable pero requiere imprimir el prompt aparte con `System.out.print()`. `System.console()` es más directo pero **puede devolver `null`** si el programa no se ejecuta desde una terminal interactiva (por ejemplo, desde un IDE o redirigiendo entrada). Eso causaría un `NullPointerException`.
- **Diferencia con C:** en C se usa `printf()` + `fgets()`. En Java, `System.console().readLine(prompt)` combina ambas cosas.

Comunicación...

▼ 5. Enviar el mensaje al servidor

```
byte[] buffer;
buffer = inputline.getBytes();
toserver.write(buffer, 0, buffer.length);
```

- `inputline.getBytes()`: convierte la cadena leída a un arreglo de bytes. Equivale a preparar el buffer que se enviará con `write()` en C.
- `toserver.write(buffer, 0, buffer.length)`:
 - **Método:** `void write(byte[] b, int off, int len)` de `DataOutputStream` (heredado de `OutputStream`).

- **Parámetros:**
 - `b`: arreglo de bytes a enviar.
 - `off`: offset desde dónde empezar a enviar (`0`).
 - `len`: cantidad de bytes a enviar (`buffer.length`).
- **Qué hace:** envía los bytes al servidor a través de la conexión TCP.
- **Diferencia con C:** en C se pasa el puntero al buffer y la cantidad de bytes. En Java se pasa el arreglo, un offset y la longitud.
- **Nota:** a diferencia de C, acá no se envía el `\0` final porque `getBytes()` no lo incluye. En C, `fgets()` sí incluye el `\n`, y se envía con `strlen(buffer)`, que tampoco incluye el `\0`.

▼ 6. Recibir la respuesta del servidor

```
buffer = new byte[256];
fromserver.read(buffer);

String resp = new String(buffer);
System.out.println(resp);
```

- `buffer = new byte[256]`: se reserva un nuevo arreglo de 256 bytes para la respuesta. Equivale a `bzero(buffer, 256)` en C.
- `fromserver.read(buffer)`:
 - **Método:** `int read(byte[] b)` de `DataInputStream` (heredado de `InputStream`).
 - **Parámetro:** `b` es el arreglo donde se guardan los bytes leídos.
 - **Valor de retorno:** cantidad de bytes leídos, o `1` si el stream llegó al final. En este ejemplo **no se captura el valor de retorno**, lo cual es un descuido: podría haber leído menos de 256 bytes y el resto quedaría con ceros.
 - **Qué hace:** lee datos del servidor. Es bloqueante si no hay datos disponibles.

- `new String(buffer)` : convierte el arreglo de bytes a una cadena usando la codificación por defecto de la plataforma.
- `System.out.println(resp)` : imprime la respuesta por pantalla. Equivale al `printf("%s\n", buffer)` del cliente C.
- **Diferencia con C:** en C se hace `bzero()` + `read()` + `printf()` . En Java, el arreglo se crea nuevo con `new byte[256]` , se lee y se convierte a `String` para imprimir.

▼ 7. Cerrar streams y socket

```
fromserver.close();
toserver.close();
socketwithserver.close();
```

- `fromserver.close()` y `toserver.close()` : cierran los streams de entrada y salida. En Java es importante cerrarlos para liberar recursos y hacer flush de los buffers.
- `socketwithserver.close()` : cierra la conexión con el servidor. Envía un `FIN` al otro extremo.
- **Orden:** primero los streams, luego el socket. Es el orden recomendado porque los streams dependen del socket.
- **Diferencia con C:** en C solo se cierra el file descriptor del socket. En Java hay que cerrar también los streams porque son objetos independientes que envuelven al socket.